Experiment No. 2

Contrast Enhancement and Histogram-Based Image Processing
using Python and OpenCV

Aim

To analyze and implement contrast enhancement techniques using histogram analysis and histogram equalization in Python and OpenCV for improving image quality and enhancing visual information in low-contrast images.

Purpose

Image contrast plays a significant role in the performance of computer vision systems. Poor illumination, sensor limitations, or environmental conditions often result in low-contrast images, making feature extraction and object recognition difficult. Histogram-based enhancement techniques improve the distribution of pixel intensities, thereby enhancing image visibility and facilitating subsequent image analysis tasks.

In this experiment, contrast stretching, histogram analysis, and histogram equalization are implemented using Python and OpenCV, and their effectiveness is evaluated on low-contrast images such as medical images, satellite images, or low-light photographs.

Prerequisites

- Basic knowledge of digital image processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, Matplotlib
- Understanding of grayscale images and pixel intensity distributions
- Jupyter Notebook, Google Colab, or Visual Studio Code

Steps (Procedure)

1. Import the required libraries (OpenCV, NumPy, Matplotlib).
2. Load a low-contrast grayscale image or convert a color image to grayscale.
3. Display the original image and generate its histogram to analyze the pixel intensity distribution.
4. Perform contrast stretching to improve the dynamic range of pixel intensities.
5. Apply Histogram Equalization using OpenCV and compare with the original image.
6. Implement CLAHE and observe its performance on images with varying illumination.
7. Plot and compare the histograms of the original, equalized, and CLAHE-enhanced images.
8. Analyze visual improvements in brightness, contrast, and feature visibility.
9. Compare the effectiveness of Histogram Equalization and CLAHE for different types of images.
10. Document the observations and discuss the role of histogram-based enhancement in computer vision.

Key Formulas

- **Contrast stretching:** $s = (r - r_{min}) / (r_{max} - r_{min}) \times (L - 1)$, with $L = 256$ for 8-bit images.
- **Histogram equalization:** $s_k = (L - 1) \times CDF(r_k) = (L - 1) \times \text{sum of } p(r_j) \text{ for } j = 0..k$, where $p(r_j) = n_j / N$.

- **CLAHE:** the image is split into tiles; each tile's histogram is clipped at *clipLimit*, the excess is redistributed, each tile is equalized, and neighbouring tiles are blended by bilinear interpolation.

Observations

- The original low-contrast image has a narrow histogram concentrated in a small intensity range, giving a low standard deviation.
- Contrast stretching expands the range linearly to 0-255 without changing the histogram's shape.
- Global histogram equalization flattens and spreads the histogram, giving the highest overall contrast, but may amplify noise and over-brighten uniform regions.
- CLAHE enhances local contrast and handles uneven illumination better, with limited noise amplification controlled by clipLimit.
- For color images, equalization should be applied to the luminance (L of LAB or V of HSV) channel only; equalizing B, G, R separately distorts colors.
- A very high clipLimit or very small tile size makes CLAHE behave noisily and produces visible tile artifacts.

Source Code

File: experiment2_histogram_processing.py

```
"""
Experiment 2: Contrast Enhancement and Histogram-Based Image Processing
Install:  pip install opencv-python numpy matplotlib
Usage:    python experiment2_histogram_processing.py [path_to_image]
"""
import os
import sys
import cv2
import numpy as np
import matplotlib.pyplot as plt

os.makedirs("output", exist_ok=True)

# ---------------------------------------------------------------
# Step 2: Load image as grayscale (or convert colour image to gray)
# ---------------------------------------------------------------
path = sys.argv[1] if len(sys.argv) > 1 else "input.jpg"
img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)

if img is None:  # fallback: synthetic low-contrast image with uneven lighting
    print(f"'{path}' not found - generating a synthetic low-contrast image.")
    base = np.zeros((300, 400), dtype=np.uint8)
    cv2.circle(base, (130, 150), 60, 120, -1)
    cv2.rectangle(base, (220, 80), (340, 220), 200, -1)
    cv2.putText(base, "CV", (150, 270), cv2.FONT_HERSHEY_SIMPLEX, 1.5, 255, 3)
    base = cv2.GaussianBlur(base, (5, 5), 0)
    gradient = np.tile(np.linspace(0.3, 1.0, 400), (300, 1))   # uneven lighting
    img = (base * gradient * 0.35 + 90).astype(np.uint8)       # squeezed to ~90-160

print(f"Image shape: {img.shape} | min={img.min()} max={img.max()} "
      f"mean={img.mean():.1f} std={img.std():.1f}")

# ---------------------------------------------------------------
# Step 3: Histogram of the original image
# ---------------------------------------------------------------
hist_orig = cv2.calcHist([img], [0], None, [256], [0, 256])

# ---------------------------------------------------------------
# Step 4: Contrast stretching (min-max linear stretch)
#   s = (r - r_min) / (r_max - r_min) * 255
# ---------------------------------------------------------------
r_min, r_max = int(img.min()), int(img.max())
stretched = ((img.astype(np.float32) - r_min) / (r_max - r_min) * 255).astype(np.uint8)
# Equivalent: cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)

# Optional: percentile stretch (robust to outliers)
p2, p98 = np.percentile(img, (2, 98))
stretched_pct = np.clip((img - p2) / (p98 - p2) * 255, 0, 255).astype(np.uint8)

# ---------------------------------------------------------------
# Step 5: Global Histogram Equalization
# ---------------------------------------------------------------
equalized = cv2.equalizeHist(img)

# ---------------------------------------------------------------
# Step 6: CLAHE (Contrast Limited Adaptive Histogram Equalization)
# ---------------------------------------------------------------
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_img = clahe.apply(img)

# ---------------------------------------------------------------
# Step 7 & 8: Compare images, histograms, and quantitative measures
# ---------------------------------------------------------------
results = {
    "Original": img,
    "Contrast Stretched": stretched,
    "Hist. Equalized": equalized,
    "CLAHE": clahe_img,
}

print("\n---- Quantitative comparison ----")
print(f"{'Method':<20}{'Min':>5}{'Max':>5}{'Mean':>8}{'Std (contrast)':>16}")
for name, im in results.items():
    print(f"{name:<20}{im.min():>5}{im.max():>5}{im.mean():>8.1f}{im.std():>16.1f}")

fig, axes = plt.subplots(3, 4, figsize=(15, 10))
for col, (name, im) in enumerate(results.items()):
    axes[0, col].imshow(im, cmap="gray", vmin=0, vmax=255)
    axes[0, col].set_title(name)
```

```
    axes[0, col].axis("off")

    h = cv2.calcHist([im], [0], None, [256], [0, 256]).ravel()
    axes[1, col].bar(range(256), h, width=1.0, color="steelblue")
    axes[1, col].set_title(f"Histogram - {name}", fontsize=9)
    axes[1, col].set_xlim(0, 255)

    cdf = np.cumsum(h) / h.sum()
    axes[2, col].plot(cdf, color="darkred")
    axes[2, col].set_title(f"CDF - {name}", fontsize=9)
    axes[2, col].set_xlim(0, 255)
    axes[2, col].set_ylim(0, 1.02)
plt.tight_layout()
plt.savefig("output/exp2_comparison.png", dpi=150)
plt.show()

# ---------------------------------------------------------------
# Step 9: HE vs CLAHE on a colour image (apply to luminance only)
#   Equalizing B, G, R separately distorts colours, so use LAB's L channel.
# ---------------------------------------------------------------
color = cv2.imread(path)
if color is not None:
    lab = cv2.cvtColor(color, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)

    he_color = cv2.cvtColor(cv2.merge((cv2.equalizeHist(l), a, b)), cv2.COLOR_LAB2BGR)
    clahe_color = cv2.cvtColor(cv2.merge((clahe.apply(l), a, b)), cv2.COLOR_LAB2BGR)

    fig, ax = plt.subplots(1, 3, figsize=(14, 4))
    for a_, im, t in zip(ax, (color, he_color, clahe_color),
                         ("Original", "HE on L channel", "CLAHE on L channel")):
        a_.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB)); a_.set_title(t); a_.axis("off")
    plt.tight_layout()
    plt.savefig("output/exp2_color.png", dpi=150)
    plt.show()

# ---------------------------------------------------------------
# Bonus: effect of CLAHE parameters
# ---------------------------------------------------------------
fig, ax = plt.subplots(1, 4, figsize=(15, 4))
for a_, (clip, grid) in zip(ax, [(1.0, (8, 8)), (2.0, (8, 8)), (4.0, (8, 8)), (2.0, (2, 2))]):
    out = cv2.createCLAHE(clipLimit=clip, tileGridSize=grid).apply(img)
    a_.imshow(out, cmap="gray", vmin=0, vmax=255)
    a_.set_title(f"clip={clip}, tiles={grid}", fontsize=9)
    a_.axis("off")
plt.tight_layout()
plt.savefig("output/exp2_clahe_params.png", dpi=150)
plt.show()
```

Questions and Answers

1. What is image contrast, and why is it important in image processing?
Contrast is the difference in intensity (or color) between the brightest and darkest regions of an image. It can be measured by the standard deviation of intensities or by $(I_{max} - I_{min}) / (I_{max} + I_{min})$.
It is important because edges, textures, and object boundaries are visible only where there is sufficient intensity difference. Low contrast hides details and degrades edge detection, segmentation, feature extraction, and recognition.

2. Explain the concept of an image histogram. What information does it provide?
An image histogram is a plot of the number of pixels (y-axis) at each intensity level (x-axis, 0-255 for 8-bit). Normalized, it is the probability distribution of pixel intensities. It provides:

- Brightness: histogram shifted left means dark image, right means bright
- Contrast: narrow histogram means low contrast, wide means high contrast
- Dynamic range, saturation or clipping (spikes at 0 or 255), and exposure problems
- A basis for thresholding (e.g., Otsu) and enhancement

3. Differentiate between Histogram Stretching and Histogram Equalization.

Aspect	Histogram (Contrast) Stretching	Histogram Equalization
Mapping	Linear: (r - r_min)/(r_max - r_min) x 255	Non-linear, based on the CDF
Histogram shape	Shape preserved, only stretched	Reshaped to be approximately uniform
Adaptivity	Fixed mapping from min and max	Adapts to the image's intensity distribution
Control	Simple, predictable, reversible	Automatic, no parameters, not reversible
Effect	Uses the full range with natural look	Stronger contrast, may over-enhance or add noise

4. What is CLAHE? How does it differ from standard Histogram Equalization?
Contrast Limited Adaptive Histogram Equalization divides the image into small tiles (e.g., 8 x 8) and equalizes each tile separately. Each tile's histogram is clipped at a limit (*clipLimit*) before computing the CDF, and the clipped excess is redistributed among all bins. Tile borders are blended by bilinear interpolation to avoid visible seams.

- **Standard HE** is global: one mapping for the whole image, so local detail in regions with different lighting may be lost.
- **CLAHE** is local and adaptive: it enhances local contrast, and the clip limit prevents over-amplification of noise in homogeneous regions.
- In OpenCV: `cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8)).apply(img)`.

5. Why is histogram equalization commonly applied before feature extraction and image segmentation?
It spreads intensities over the full range, making edges, corners, and textures more distinct and consistent across images with different lighting. This gives stronger gradient responses for edge and feature detectors, makes histogram-based thresholding (e.g., Otsu) more reliable because classes separate better, and reduces the effect of illumination variation, so features and segments are more stable and reproducible.

6. Mention three real-world applications where histogram equalization is widely used.

1. Medical imaging: enhancing X-ray, CT, MRI, and mammography images
2. Satellite and remote sensing: improving hazy or low-contrast aerial images
3. Low-light and surveillance photography, and face recognition preprocessing (also license plate recognition and OCR of faded documents)

7. What are the limitations of global histogram equalization?

- Treats the whole image uniformly, so it fails on images with uneven or mixed lighting
- Amplifies noise, especially in near-uniform regions
- Can over-enhance, causing washed-out or unnatural appearance and loss of detail in very bright or dark areas
- Changes the mean brightness, which is undesirable where brightness must be preserved
- Applied per-channel on color images, it distorts colors
- Not reversible and has no parameter to control the strength

8. How does contrast enhancement improve the performance of object detection and recognition systems?

Enhanced contrast makes object boundaries, textures, and shapes more distinct, so detectors find more reliable edges and keypoints. It reduces missed detections in dark or hazy scenes, normalizes inputs taken under different lighting conditions (improving robustness and generalization of classifiers and CNNs), and improves segmentation and feature matching, leading to higher accuracy, precision, and recall.

9. Compare histogram-based enhancement techniques with brightness adjustment methods.

Aspect	Brightness adjustment	Histogram-based enhancement
Operation	Adds or subtracts a constant: g = f + b	Remaps intensities using the histogram or CDF
Effect on histogram	Shifts it left or right; shape and width unchanged	Spreads or reshapes it to use the full range
Contrast	No real change (may clip at 0 or 255)	Improves contrast
Adaptivity	Same for all pixels, manual parameter	Data-driven, automatic (HE) or local (CLAHE)
Use	Simple lightening or darkening	Low-contrast, hazy, or poorly exposed images

10. Why is CLAHE preferred for medical imaging and low-light image enhancement?

- Medical images (X-ray, MRI, CT) have subtle local intensity differences that matter diagnostically; CLAHE enhances them in each region without washing out the whole image.
- The clip limit prevents noise amplification in homogeneous tissue or dark areas, which is a major problem with global HE.
- It handles uneven illumination, as in low-light scenes with both very dark and bright regions.
- Tile-wise processing with interpolation preserves natural appearance and avoids over-enhancement and visible seams.
- It is parameterized (clipLimit, tileGridSize), so the strength can be tuned for the modality.

Conclusion

Contrast stretching, global histogram equalization, and CLAHE were implemented using Python and OpenCV and compared using images, histograms, and quantitative statistics. Contrast stretching is simple and preserves the histogram shape, global equalization gives strong automatic enhancement but may amplify noise and over-enhance, and CLAHE gives the best results for images with uneven illumination, making it the preferred choice in medical and low-light imaging.